Chapter 11 — KV Cache: The Heart of LLM Serving

> *"The KV cache is not an implementation detail. It is the central resource constraint that determines how many users your system can serve, how fast they receive their first token, and how much each request costs you. Every inference engineer must know its math cold."* — Production LLM Serving Principle

---

Chapter Overview

When a user sends a 2,000-token prompt to a 70B model, the GPU does not run the model once. It runs prefill — an $O(S)$ to $O(S^2)$ attention computation over the entire prompt — then enters decode mode, generating tokens one at a time. Each decode step requires attending to every prior token. Without caching the Key and Value tensors computed during prefill, every new token would require recomputing attention from scratch: quadratic in sequence length, catastrophically slow.

The Key-Value cache is the solution. It trades HBM memory for computation time. But HBM is the scarcest resource in any GPU cluster. Managing that trade-off — how much to allocate, how to page it, how to compress it, how to reuse it across requests, and how to tier it across PCIe and NVMe when pressure mounts — is the defining engineering challenge of production LLM inference.

Sections in this chapter:

- 11.1 KV Cache Memory Math — Derivation and Capacity Planning
- 11.2 PagedAttention — Virtual Memory for the KV Cache
- 11.3 KV Cache Quantization — FP8, INT8, and INT4
- 11.4 Prefix Caching — Eliminating Redundant Prefill
- 11.5 KV Cache Tiering — CPU DRAM and NVMe Overflow
- 11.6 Disaggregated Prefill-Decode and KV Transfer
- 11.7 Multi-Head Latent Attention (MLA) — DeepSeek's Compression Architecture [r2]
- 11.8 KV Cache Decision Framework — ROI Ranking and Anti-Patterns
- 11.9 Key Takeaways and Review Questions

11.1 KV Cache Memory Math

11.1.1 Why the KV Cache Exists

In autoregressive decoding, every output token must attend to every preceding token. The Key and Value projections for those preceding tokens are deterministic functions of the input: given the same token sequence, the same K and V tensors result every time. Recomputing them at every decode step is correct but wasteful:

```
DECODE COMPUTE WITH AND WITHOUT KV CACHE:

  WITHOUT KV CACHE (recompute every step):
    Generate token 101:
      Prefill all 100 prior tokens → compute K,V for each → attention → output
      FLOPs ≈ 2 × N_params × 100   (all prior tokens re-attended)
    Generate token 102:
      Prefill all 101 prior tokens → compute K,V for each → attention → output
      FLOPs ≈ 2 × N_params × 101
    ...
    Generate token N:
      FLOPs ≈ 2 × N_params × (N-1)
    Total FLOPs ∝ O(N²)    (quadratic in generation length)

  WITH KV CACHE (incremental):
    Generate token 101:
      Load K,V for tokens 1-100 from cache → compute K,V for token 101 → attention
      FLOPs ≈ 2 × N_params × 1   (one new token only)
    Generate token 102:
      Load K,V for tokens 1-101 from cache → compute K,V for token 102 → attention
      FLOPs ≈ 2 × N_params × 1   (still one new token)
    Total FLOPs ∝ O(N)     (linear in generation length)

  The trade-off is explicit and exact:
    Without KV cache: O(N²) compute, O(1) memory per token
    With KV cache:    O(N) compute,   O(N) memory per token
  At N=1000 tokens: cache avoids 1000× more compute at cost of N×KV memory.
```

11.1.2 KV Cache Size Formula

```
KV CACHE SIZE — FIRST PRINCIPLES:

  Per-layer KV cache for one sequence:
    KV_bytes_per_layer = 2 × S × H_kv × d_head × bytes_per_element
```

```
    2            = K tensor + V tensor
    S            = current sequence length (tokens)
    H_kv         = number of KV heads (GQA: often 8; MHA: H_q)
    d_head       = per-head dimension (typically 128)
    bytes        = 2 (BF16), 1 (FP8/INT8), 0.5 (INT4)

  Total KV cache for one sequence:
    KV_total = KV_bytes_per_layer × L   (L = number of layers)

LLAMA-3 70B (GQA-8, d_head=128, L=80) — REFERENCE TABLE:

  Context   BF16 bytes/tok/layer  BF16 per seq   FP8 per seq   Heads
  ──────────────────────────────────────────────────────────────────────
  1K tok    2×8×128×2 = 4,096 B   328 MB         164 MB        GQA-8
  4K tok                           1,310 MB       655 MB
  8K tok                           2,621 MB      1,310 MB
  32K tok                          10,486 MB     5,243 MB
  128K tok                         41,943 MB    20,972 MB
  ──────────────────────────────────────────────────────────────────────


  BF16 per token per layer: 2 × 8 × 128 × 2 = 4,096 bytes = 4 KB
  FP8  per token per layer: 2 × 8 × 128 × 1 = 2,048 bytes = 2 KB
  INT4 per token per layer: 2 × 8 × 128 × 0.5 = 1,024 bytes = 1 KB

CRITICAL NUMBERS (memorize):
  LLaMA-3 70B BF16, S=4096: 1.34 GB per sequence
  LLaMA-3 70B FP8,  S=4096: 0.67 GB per sequence
  LLaMA-3 70B BF16, S=128K: 41.9 GB per sequence (fills half an H100!)
```

11.1.3 KV vs Weight Memory: The HBM Budget Split

```
H100 SXM5 (80 GB) — MEMORY BUDGET FOR 70B SERVING:

  Configuration         Weights    KV Budget    Max concurrent seqs (S=4096)
  ──────────────────────────────────────────────────────────────────────────
  BF16 weights, TP=2    140/2=70 GB   10 GB     10 / 1.34 = 7 seqs/GPU
  FP8 weights, TP=1      70 GB        10 GB     10 / 0.67 = 14 seqs
  FP8 weights + FP8 KV   70 GB        10 GB     10 / 0.67 = 14 seqs (same — FP8 KV
is KV budget calc)
  INT4 weights, TP=1     35 GB        45 GB     45 / 0.34 = 132 seqs (INT4 KV,
S=4096)
  ──────────────────────────────────────────────────────────────────────────
```

```
KV BUDGET RULE OF THUMB:
  KV_budget_GB = (GPU_HBM_GB × 0.85) - weights_GB_per_GPU
  Safety factor 0.85 leaves room for activations, CUDA overhead, page tables.

For H100, FP8 weights (TP=1, 70 GB):
  KV_budget = 80 × 0.85 - 70 = 68 - 70 = -2 GB  ← negative! Barely fits.
  Use GPU_utilization=0.90:  72 - 70 = 2 GB KV  ← very tight
  Realistic: serve single-request context only or use TP=1 with small KV

For H200, FP8 weights (TP=1, 70 GB):
  KV_budget = 141 × 0.85 - 70 = 119.9 - 70 = 49.9 GB
  At FP8 KV, S=4096: 49.9 / 0.67 = 74 concurrent sequences!
  H200 is the sweet spot for 70B production serving.

The numbers reveal why FP8 weights matter beyond throughput:
  BF16 weights barely fit on H200 (1 GB KV headroom).
  FP8 weights leave 50 GB for KV — dramatically changes serving capacity.
```

11.1.4 Five Reference Numbers to Know Cold

```
REFERENCE KV SIZES (LLaMA-3 70B GQA-8, FP8):

  #1  Per token per layer:     2 KB         (2 × 8 × 128 × 1B)
  #2  Per token all layers:    160 KB        (2 KB × 80 layers)
  #3  Per sequence at 4K ctx:  0.67 GB      (160 KB × 4,096 tokens)
  #4  Per sequence at 32K ctx: 5.12 GB      (160 KB × 32,768 tokens)
  #5  Per sequence at 128K ctx: 20.5 GB     (160 KB × 131,072 tokens)

  These five numbers answer 90% of capacity planning questions instantly.
  Linear interpolation for other context lengths: KV_GB = tokens × 160 KB / 1e9
```

11.1.5 Worked Example: Serving Capacity Planning [r2]

This worked example shows how to dimension an H100 cluster for a production chatbot with a known traffic profile. Every step maps directly to a production planning decision.

```
PROBLEM STATEMENT:
  Model:           LLaMA-3 70B
  Hardware:        H100 SXM5 (80 GB), up to 16 GPUs available
  Traffic target:  1,000 concurrent users
  Avg context:     4K tokens (system prompt 2K + avg conversation)
  Peak context:    32K tokens (5% of sessions)
  Weights:         FP8 (70 GB total; TP=2 → 35 GB per GPU)
  KV cache:        FP8

STEP 1: KV PER SESSION
  FP8 KV at 4K ctx:  4,096 × 160 KB = 655 MB = 0.64 GB
  FP8 KV at 32K ctx: 32,768 × 160 KB = 5,243 MB = 5.12 GB

STEP 2: KV BUDGET PER GPU (TP=2 setup, H100 pair)
  Per H100: 80 GB total
  FP8 weights: 70 GB / 2 = 35 GB per GPU
  Framework overhead: ~3 GB
  KV budget per GPU: 80 - 35 - 3 = 42 GB per GPU
  KV budget per TP=2 pair: 2 × 42 = 84 GB

STEP 3: CONCURRENT SESSIONS AT 4K AVG CTX
  Sessions per TP=2 pair: floor(84 GB / 0.64 GB) = 131 sessions
  With 80% safety (leave headroom): 131 × 0.80 = 104 sessions per pair

STEP 4: GPU PAIRS NEEDED FOR 1,000 USERS
  Pairs needed: ceil(1,000 / 104) = 10 GPU pairs = 20 H100s
  We have 16 H100s (8 pairs): serves 8 × 104 = 832 concurrent users
  To serve 1,000 users: need 20 H100s (within budget if we have 20 total)

STEP 5: HANDLING 32K PEAK SESSIONS (5% of 1,000 = 50 sessions)
  KV at 32K: 5.12 GB per session
  50 sessions: 50 × 5.12 = 256 GB KV needed for long-context users
  8 GPU pairs: 8 × 84 = 672 GB total KV budget
  KV for short users (950 at 4K): 950 × 0.64 = 608 GB
  KV for long users (50 at 32K):  50 × 5.12 = 256 GB
  Total: 608 + 256 = 864 GB > 672 GB available
  → 16 H100s cannot serve 1,000 users with 5% at 32K context simultaneously.

STEP 6: HYBRID TIER SOLUTION
  Route long-context (>8K) to a dedicated tier.
  Short-context tier (16 GPUs, 8 pairs): 8 × 104 = 832 users at avg 4K
  Long-context tier: need ceil(50 × 5.12 / 84) = ceil(3.05) = 4 more GPUs (2
pairs)
```

```
  Total: 18 H100s serves 950 short + 50 long = 1,000 users

CAPACITY TABLE (16 H100s, FP8 weights, FP8 KV):
  Avg context   Concurrent users   Bottleneck
  ──────────────────────────────────────────
  1K tokens     1,600+             GPU compute
  4K tokens     832                KV memory
  8K tokens     416                KV memory
  32K tokens    128                KV memory
  128K tokens    32                KV memory
  ──────────────────────────────────────────
  Rule: each 2× increase in context halves concurrent capacity.
  This is the linear trade-off: concurrency ∝ 1/context_length.
```

11.1.6 Worked Example: Prefix Cache Hit Rate Economics [r2]

```
PROBLEM: Production chatbot, 1,000,000 requests/day
  System prompt: 2,048 tokens (fixed for all requests)
  User query: ~200 tokens (variable)
  Cache hit rate: 80%

STEP 1: TOKENS SAVED PER DAY

  WITHOUT prefix cache:
    Prefill tokens/request: 2,048 + 200 = 2,248 tokens
    Total tokens/day: 1,000,000 × 2,248 = 2.248 billion tokens

  WITH prefix cache (80% hit rate):
    Cache hits (800K requests): only prefill user query = 200 tokens
    Cache misses (200K requests): full prefill = 2,248 tokens
    Total tokens/day: 800,000 × 200 + 200,000 × 2,248 = 609.6 million tokens

    Tokens saved: 2,248M - 610M = 1,638M = 1.64 billion tokens/day saved
    Reduction: 72.8% fewer prefill tokens

STEP 2: GPU-HOURS SAVED

  Prefill throughput (H100, FP8, batch=8): ~120,000 tokens/sec

  Without cache: 2.248B / 120,000 = 5.20 GPU-hours for prefill
  With cache:    0.610B / 120,000 = 1.41 GPU-hours for prefill
  GPU-hours saved: 3.79 GPU-hours/day
```

```
  At $1.66/GPU-hour (owned H100): $6.29 saved per day in compute alone

STEP 3: TTFT IMPROVEMENT

  Without cache: TTFT ∝ 2,248 tokens to prefill → ~380ms
  With cache (hit): TTFT ∝ 200 tokens to prefill  → ~34ms
  TTFT reduction on cache hits: 89% faster first token!

STEP 4: KV MEMORY FREED

  Without cache: 1,000 concurrent users × 2,248 tokens × 0.16 MB/tok = 360 GB KV
  With cache:    1,000 users with only user query active in personal KV:
                 1,000 × 200 tokens × 0.16 MB/tok = 32 GB personal KV
                 1 shared system prompt copy: 2,048 × 0.16 MB × 1 = 328 MB
                 Total: ~32.3 GB KV (vs 360 GB without cache)
  KV memory freed: 360 - 32.3 = 328 GB = 91% less KV memory!

STEP 5: BUSINESS CASE SUMMARY TABLE

  Metric                      Without Cache   With 80% Hit Rate   Savings
  ──────────────────────────────────────────────────────────────────────────
  Prefill tokens/day          2.248 billion   0.610 billion       72.8%
  GPU-hours for prefill/day   5.20            1.41                73.3%
  KV memory (1K users)        360 GB          32.3 GB             91.0%
  Concurrent capacity         1,000 users     ~10,800 users       980% ↑
  TTFT (on cache hit)         380ms           34ms                89%
  Daily compute cost          $8.63           $2.34               $6.29
  Annual compute cost         $3,150          $854                $2,296
  ──────────────────────────────────────────────────────────────────────────

  The concurrent capacity multiplier (11×) is the most important number:
    For the same 16 H100s, 80% cache hit rate serves 11× more users —
    because each cached user carries only their 200-token unique query
    in active KV, not the 2,048-token system prompt.

IMPLEMENTATION REFERENCE:
  vLLM: --enable-prefix-caching (default since v0.4.0)
  SGLang: RadixAttention is default (block_reuse=True)
  TRT-LLM: kv_cache_config.enable_block_reuse = True

  Measure hit rate in production:
    curl http://localhost:8000/metrics | grep prefix_cache_hit_rate
    # vllm:cpu_prefix_cache_hit_rate{...} 0.78
```

11.2 PagedAttention — Virtual Memory for the KV Cache

11.2.1 The Fragmentation Problem

Before PagedAttention (vLLM, 2023), serving systems pre-allocated KV cache as a contiguous block per request: `max_seq_len × KV_per_token` bytes reserved at admission.

```
CONTIGUOUS KV ALLOCATION — THE WASTE:

  max_seq_len = 4,096 tokens (supported context)
  KV_per_token (FP8) = 160 KB
  Pre-allocated per request: 4,096 × 160 KB = 640 MB per request

  Actual traffic distribution:
    50% of requests: < 200 tokens
    25% of requests: 200–1,000 tokens
    20% of requests: 1,000–3,000 tokens
     5% of requests: 3,000–4,096 tokens

  Average actual usage: ~800 tokens × 160 KB = 128 MB
  Reserved per request: 640 MB
  Fragmentation waste: (640 - 128) / 640 = 80%!

  At batch=20 (all pre-allocated):
    HBM reserved: 20 × 640 MB = 12.8 GB
    Actual KV data: 20 × 128 MB = 2.56 GB
    Wasted: 10.2 GB = 80% of KV allocation

  With 40 GB KV budget: only 8 GB holds real data.
  Effective capacity: 5× lower than the hardware physically supports.

  Every other optimization — quantization, GQA — is undermined when
  the allocation policy wastes 4× more memory than the data requires.
```

11.2.2 Page Table Mechanics

```
PAGED KV CACHE MECHANICS:

PHYSICAL HBM LAYOUT:
  Total KV budget divided into fixed-size PHYSICAL BLOCKS.
  Each block holds: block_size × H_kv × d_head × 2 (K+V) × bytes_per_element
```

```
  For block_size=16, LLaMA-3 70B FP8, one layer:
    Block = 16 × 8 × 128 × 2 × 1 = 32,768 bytes = 32 KB per block
    All 80 layers: 32 KB × 80 = 2.5 MB per physical block group

  40 GB KV budget / 2.5 MB per block group = 16,000 physical block groups

PER-REQUEST BLOCK TABLE (the page table):
  Logical sequence positions → physical block IDs

  Request A (500 tokens, 32 blocks used):
    block_table[0]  → physical_block_#1,247  (positions 0-15)
    block_table[1]  → physical_block_#5,823  (positions 16-31)
    ...
    block_table[31] → physical_block_#91,004 (positions 496-511)

  Physical blocks are NON-CONTIGUOUS — scattered anywhere in HBM.
  The block table is tiny (one integer per logical block) in CPU memory.

ALLOCATION POLICY:
  Admission: allocate 0 blocks (lazy)
  As sequence grows into new block: allocate 1 physical block
  At completion: free ALL blocks immediately

  No pre-reservation. No internal fragmentation.
  HBM holds only KV data that actually exists.

FRAGMENTATION COMPARISON:
  Contiguous pre-allocation: 80% internal fragmentation (avg 800/4096 tokens)
  PagedAttention: last block is partially filled; expected waste = (block_size-
1)/2
  Average waste per sequence: 7.5 tokens / 800 avg tokens = 0.9%
  80% → 0.9%: 89× reduction in KV memory waste.
  Effective concurrent users: ~5× more for the same HBM.

ATTENTION WITH BLOCK TABLE (the kernel side):
  Standard FlashAttention: assumes contiguous K,V in memory
  PagedAttention kernel: takes block_table as input; fetches K,V blocks by
reference
  Cost: one extra indirection per block in the attention kernel (~3% overhead)
  Net: 5× more capacity for 3% compute cost — always the right trade.
```

11.2.3 Copy-on-Write for Prefix Sharing

```
COPY-ON-WRITE (CoW) — PHYSICAL BLOCK SHARING:

  Use case: multiple sequences sharing a common prefix (e.g., system prompt).
  Without CoW: each sequence has its own copy of the prefix blocks.
  With CoW: all sequences SHARE the same physical blocks for the shared prefix.

  Mechanism:
    Shared prefix blocks: ref_count > 1
    When a sequence needs to WRITE to a shared block (beam search divergence,
    or when a request appends a new token after the shared prefix):
      Allocate a NEW physical block
      Copy the shared block to the new block
      Update the requesting sequence's block_table to point to the new block
      Decrement ref_count on the original block

    Read-only (most decode steps): access shared blocks directly — no copy.

  SAVINGS (1,000 users, 2,048-token system prompt, 16-token blocks):
    Unique prefix blocks: ceil(2,048 / 16) = 128 blocks × 2.5 MB = 320 MB
    Without CoW: 1,000 × 320 MB = 320 GB for system prompt copies
    With CoW:    1 × 320 MB = 320 MB shared (+ small individual suffixes)
    Savings: 319.7 GB = 99.9%!

    This is the physical-block implementation of prefix caching (§11.4).
    PagedAttention's CoW mechanism is what makes prefix caching efficient.
```

11.3 KV Cache Quantization

11.3.1 Why KV Quantization Is Relatively Safe

```
KV QUANTIZATION STABILITY ANALYSIS:

  Unlike weight quantization (error accumulates across 80 layers),
  KV quantization error affects only the current attention step.

  Why the attention mechanism is quantization-robust:
    1. Softmax normalization: attention scores exponentiated and normalized.
       Small absolute errors in K,V produce small relative errors in weights.
    2. Multi-head averaging: 8 KV heads (GQA), each with independent KV values.
```

```
       Errors in one head are diluted by other heads in the output.
    3. Per-token granularity: quantize at per-token scale (not per-tensor).
       Each token's K,V has its own scale factor → captures local magnitude
variation.

KV QUANTIZATION REFERENCE (LLaMA-3 70B):

  Format   Quality loss (MMLU)  Memory reduction  Concurrent users  Implementation
```

Format	Quality loss (MMLU)	Memory reduction	Concurrent users	Implementation
BF16	0% (baseline)	1×	baseline	Default
FP8	-0.2%	2×	~2×	vLLM, TRT-LLM
INT8	-0.5%	2×	~2×	vLLM, TRT-LLM
INT4	-1.5%	4×	~4×	Experimental

```
  RECOMMENDATION: Always deploy FP8 KV in production.
    -0.2% quality loss is within measurement noise for most applications.
    2× more concurrent users at zero hardware cost.
    One config flag: vLLM --kv-cache-dtype fp8_e5m2

  INT4 KV: only after careful evaluation on your specific task.
    -1.5% MMLU is meaningful for knowledge-intensive applications.
    Acceptable for: casual chat, summarization of recent events.
    Caution for: coding, math, factual QA, multi-step reasoning.
```

11.3.2 Per-Token KV Quantization — The Correct Granularity

```
KV QUANTIZATION GRANULARITY:

PER-TENSOR (wrong for KV — don't use):
  One scale for the entire K or V tensor.
  Problem: Key magnitudes vary 10-100× across different positions.
           Early tokens (context) have different statistics than recent tokens.
           One scale is dominated by outlier positions.
  Quality: poor (equivalent to INT4 weight per-tensor failure case).

PER-CHANNEL (per head — better but not standard):
  One scale per KV head.
  Captures head-level magnitude differences.
  Used by some INT8 implementations.
```

```
PER-TOKEN (correct for KV — production standard):
  One scale per token position.
  q_scale[t] = max(|K[t,:]|) / 127   (INT8)
  Q(K[t,:]) = round(K[t,:] / q_scale[t])

  Why this works: K and V tensors grow as generation proceeds.
  Earlier positions have seen more attention updates than recent positions.
  Per-token scale captures this temporal variation.

  Memory overhead: 1 scale per token per layer
    = 1 float × S × L × 2 bytes = 2SL bytes
    At S=4096, L=80, FP16 scale: 2 × 4096 × 80 × 2 = 1.3 MB
    vs 320 MB for the FP8 KV data itself → negligible overhead.

FP8 KV CACHE — VLLM CONFIGURATION:
  # vLLM server startup
  python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3-70b-instruct \
    --kv-cache-dtype fp8_e5m2 \          # FP8 E5M2 for KV (wider range for
gradients)
    --gpu-memory-utilization 0.90 \
    --tensor-parallel-size 2             # TP=2 for 70B on H100 BF16, or TP=1 with
FP8

  # TRT-LLM configuration
  trtllm-build \
    ... \
    --kv_cache_type paged \
    --kv_cache_quant_algo FP8 \
    --kv_cache_quant_granularity TOKEN
```

11.4 Prefix Caching — Eliminating Redundant Prefill

11.4.1 When Prefill Can Be Skipped

Prefix caching saves prefill compute when multiple requests share a common prefix — most commonly a system prompt, a RAG retrieval context, or a multi-turn conversation history that doesn't change between turns.

```
WORKLOADS THAT BENEFIT MOST:

  Workload type          Shared prefix?  Typical hit rate  TTFT reduction
  ──────────────────────────────────────────────────────────────────────────
  Chatbot (fixed system prompt)    Yes   70-90%           50-89%
  RAG with fixed document          Yes   80-95%           60-90%
  Multi-turn conversation          Yes   50-85%           40-70%
  Agent with tool descriptions     Yes   85-95%           70-90%
  Single-turn Q&A (no prompt)      No    < 5%             < 5%
  Creative writing (varied prefix) No    < 10%            < 10%
  ──────────────────────────────────────────────────────────────────────────

  RULE OF THUMB: Any workload with a constant or slowly-changing prefix
  longer than 256 tokens benefits substantially from prefix caching.
  The benefit scales with: (prefix_length / total_length) × hit_rate.
```

11.4.2 Flat Hash-Based Prefix Caching (vLLM)

```
FLAT HASH CACHE ALGORITHM:

  Cache structure: hash_table[block_hash] → physical_block_id

  Cache key construction (per block):
    block_0_key = SHA-256(token_ids[0:16])            [tokens 0-15]
    block_1_key = SHA-256(token_ids[0:32])            [tokens 0-31 — prefix-
committing]
    block_2_key = SHA-256(token_ids[0:48])            [tokens 0-47]
    ...
    Each key commits to the ENTIRE prefix through that block position.
    This ensures: block at position t is only shared if ALL prior blocks match.

  Cache lookup on new request:
    Walk token_ids in 16-token steps from position 0.
    For each block: look up block_key in hash_table.
    Find the LONGEST CONTIGUOUS prefix where ALL blocks hit cache.
    Stop at first miss — cannot use cached block B if block B-1 missed.
    Reuse hit blocks (shared read-only physical pages via CoW).
    Only prefill the MISS portion.

  Cache key requirements:
    Must include model_id (FP8 KV ≠ BF16 KV for same tokens)
    Must include lora_adapter_id (adapters change activations → different KV)
```

```
    SHA-256 is overkill; shorter hash acceptable in practice.

  Eviction policy: LRU (Least Recently Used)
    Evict blocks with oldest last_access timestamp when KV pool near capacity.
    Protects recently-accessed prefixes from eviction.

  Limitation: flat structure stores prefix blocks independently per request.
    Request A: system_prompt + query_A → stores hash(system_prompt ++
query_A[:16])
    Request B: system_prompt + query_B → stores hash(system_prompt ++
query_B[:16])
    The system_prompt portion is referenced by both but NOT stored as a single
shared node.
```

11.4.3 RadixAttention — Tree-Structured Prefix Sharing (SGLang)

```
RADIXATTENTION (SGLang, 2024) — TREE STRUCTURE:

  Problem with flat caching:
    Request A: [system_prompt=1024 tok] + [query_A=128 tok]
    Request B: [system_prompt=1024 tok] + [query_B=128 tok]

    Flat cache: system_prompt blocks referenced separately per request.
    System_prompt blocks are NOT physically shared — duplicated internally.

  Radix Tree solution:
    root
    └── [system_prompt blocks: physical #100-163]   ref_count=N
        ├── [query_A blocks: #164-171]               ref_count=1
        └── [query_B blocks: #172-179]               ref_count=1

    System_prompt = ONE set of physical blocks, shared by all N requests.
    Memory: 1× system_prompt + N × unique_query  (vs N× without tree)

  LOOKUP ALGORITHM:
    New request → walk radix tree from root.
    At each node: does next block match?
      YES → descend (cache hit, share physical blocks)
      NO  → fork (allocate new node for this divergent path)
    O(S / block_size) lookup time.

  EVICTION POLICY — LEAVES FIRST:
    Evict leaf nodes (no active children) with lowest last_access.
```

```
    Evicting a leaf: free physical blocks, decrement parent ref_count.
    Parent evicted only after all children evicted.
    This ensures: shared prefixes (inner nodes) outlive unique suffixes (leaves).
    Shared system prompt stays in cache as long as ANY request uses it.

  PERFORMANCE vs FLAT HASH (SGLang benchmarks, mixed chat workload):
    Prefix cache hit rate:     72-89%  (vs 60-80% flat)
    Prefill compute reduction: proportional to hit_rate × prefix_len / total_len
    Throughput improvement:    1.5-3× vs no prefix caching

  HIT RATE ECONOMICS FORMULA:
    cost_reduction = 1 - (prefix × (1 - hit_rate) + query_len) / (prefix +
query_len)

    At hit_rate=0.80, prefix=2048, query=200:
      cost_reduction = 1 - (2048×0.2 + 200) / 2248 = 1 - 609.6/2248 = 72.9%

    At hit_rate=0.95 (warm steady state):
      cost_reduction = 1 - (2048×0.05 + 200) / 2248 = 1 - 302.4/2248 = 86.5%
```

11.4.4 Multi-Turn Conversation Prefix Caching

```
MULTI-TURN CACHE CHAINING:

  Turn 1: [system_prompt][user_1][assistant_1]     → cache after response
  Turn 2: [system_prompt][user_1][assistant_1][user_2][assistant_2]
  Turn 3: [system_prompt][user_1]...[user_3][assistant_3]

  With prefix caching:
    Turn 2 prefill: only [user_2] is new (hits cache for everything prior)
    Turn 3 prefill: only [user_3] is new

    Each turn's prefill cost: proportional only to the NEW tokens.
    For a 10-turn conversation with 200 tokens/turn:
      Without cache: 200 + 400 + 600 + ... + 2000 = 11,000 prefill tokens
      With cache:    200 + 200 + 200 + ... + 200 = 2,000 prefill tokens
      Savings: 81.8% for 10-turn conversation!

  IMPLEMENTATION NOTE — STICKY ROUTING:
    For multi-turn prefix caching to work, subsequent turns MUST route to
    the same server instance (the one holding that conversation's KV cache).
    Use: session-ID-based routing at the load balancer.
    Challenge: long conversations accumulate large per-session KV → memory
```

```
pressure.
    Solution: LRU eviction with session-level priority (active sessions
protected).
```

11.5 KV Cache Tiering — CPU DRAM and NVMe Overflow

When HBM KV capacity is exhausted, the scheduler has three choices: reject the request (return 429), evict an existing sequence (recompute on re-admission), or offload to a slower tier. Tiering enables higher peak concurrency at the cost of additional latency when the GPU tier fills.

```
KV CACHE TIER HIERARCHY:

  Tier 1 — GPU HBM:
    Bandwidth: 3.35–8 TB/s (H100–B200)
    Latency for attention: nanoseconds (on-chip)
    Capacity: 10–50 GB (after weights)
    Cost: highest

  Tier 2 — CPU DRAM (via PCIe / NVLink-C2C):
    Bandwidth: 64–128 GB/s (PCIe 5.0 x16, bidirectional)
    Latency for swap: 2–10 ms (DMA transfer)
    Capacity: 512 GB – 2 TB (host server RAM)
    Cost: moderate (standard server DRAM)
    Capacity multiplier: 10–40× more than HBM tier

  Tier 3 — NVMe SSD (via CPU PCIe):
    Bandwidth: 6–12 GB/s (PCIe Gen5 NVMe)
    Latency for swap: 50–500 ms
    Capacity: 4–32 TB (NVMe SSD fleet)
    Cost: lowest per GB
    Capacity multiplier: 100–1000× vs HBM

  BANDWIDTH BUDGET FOR TIER 2 SWAP:
    Swap size per sequence (4K ctx, FP8): 0.67 GB
    PCIe 5.0 bandwidth (GPU → CPU): 64 GB/s (x16, unidirectional)
    Swap time: 0.67 GB / 64 GB/s = 10.5 ms latency penalty

    In TTFT budget (target 300 ms): 10.5 ms = 3.5% → acceptable
    In TPOT budget (target 30 ms):  10.5 ms = 35% → unacceptable for active decode

    RULE: Tier 2 swapping is acceptable for INACTIVE sequences (queued/waiting).
          Never swap an ACTIVE decode sequence — adds 35% TPOT penalty.
          Use swap for: prefill queue, multi-turn sessions between turns.

KV TIERING DECISION POLICY:
  Active decode:   Always in HBM (never swap during active generation)
  Completed turn:  Eligible for tier 2 after response sent
  Long-idle (>5min): Eligible for tier 3 or eviction
  New request with shared prefix: Load from tier 2/3 before prefill starts

  vLLM implementation: --swap-space 32  (GB of CPU DRAM for KV swap)
  TRT-LLM: host_cache_size parameter in kv_cache_config
```

11.6 Disaggregated Prefill-Decode and KV Transfer

Chapter 6 introduced disaggregated P-D from the serving architecture perspective. This section focuses specifically on the KV transfer mechanics.

```
KV TRANSFER PIPELINE (NVIDIA Dynamo / NIXL protocol):

  NAIVE KV TRANSFER (NCCL):
    After prefill GPU generates KV cache → NCCL AllReduce to transfer to decode
GPU
    Problem: NCCL is designed for collective ops; P2P transfer is not its
strength.
    Effective bandwidth: 15–20 GB/s
    For LLaMA-3 70B, 4K ctx, INT8 KV: 2.7 GB / 15 GB/s = 180 ms transfer

  NIXL OPTIMIZED TRANSFER (NVIDIA Dynamo 2025):
    RDMA with pre-registered GPU memory:
      GPU KV blocks pre-registered with IB NIC → no per-transfer pinning overhead
      Batched RDMA writes: multiple block transfers coalesced → higher efficiency
    Layer-by-layer pipeline:
      Transfer KV for layer l   | Decode attention for layers 1..l-1
      Transfer KV for layer l+1 | Decode attention for layer l
      ...
      KV transfer overlap with decode compute → lower observable latency

    Effective bandwidth: 35–45 GB/s (2× NCCL)
    Overlap factor: ~70% of transfer hidden behind compute
    Observable latency: 2.7 GB / (40 GB/s / 0.3) = 20 ms (vs 180 ms naive)

  KV TRANSFER BANDWIDTH REQUIREMENTS:
    LLaMA-3 70B, INT8 KV, 4K ctx, 100 RPS:
      KV per request: 2.7 GB
      Transfers/sec: 100
      Required bandwidth: 100 × 2.7 GB/s = 270 GB/s
      InfiniBand NDR (50 GB/s × 8 ports): 400 GB/s ← sufficient

    At 1,000 RPS:
      Required bandwidth: 2,700 GB/s → not achievable with standard IB
      Need: aggregated high-bandwidth fabric or reduced KV (FP8, GQA)
      With FP8 KV: 2,700/2 = 1,350 GB/s → requires 27× IB NDR ports per node

  WHEN P-D DISAGGREGATION IS WORTH THE COMPLEXITY:
    Fleet > 100 GPUs (infrastructure complexity amortized)
    Avg prompt > 1,024 tokens (prefill-heavy workload)
    Avg output > 500 tokens (decode dominates, worth optimizing hardware)
    TTFT SLA > 300 ms (room for KV transfer latency)
    Cost optimization priority (batch/API, not interactive)
```

11.7 Multi-Head Latent Attention (MLA) — DeepSeek's KV Compression [r2]

11.7.1 The Core Insight

GQA reduces KV memory by using fewer heads — fewer unique K,V projections means fewer bytes to cache. MLA takes a fundamentally different approach: use full-rank K and V at attention time, but cache only a compressed low-dimensional latent representation. Reconstruct K,V from the latent on the fly using fixed weight matrices.

The result: KV memory approaching MQA levels with quality approaching full MHA.

11.7.2 The Low-Rank Compression Architecture

```
MLA MATHEMATICAL FORMULATION (DeepSeek-V3):

Standard MHA (reference):
  K_h = x × W_K_h   (W_K_h ∈ R^{d_model × d_head})  for each head h
  V_h = x × W_V_h
  KV cache stores: {K_1,...,K_H, V_1,...,V_H} per token
  Bytes per token per layer = 2 × H × d_head × P
  DeepSeek-V3 (H=128, d_head=128, BF16): 2×128×128×2 = 65,536 bytes = 64 KB

MLA STEP 1 — Down-project to latent vector c_KV:
  c_KV = x × W_DKV    (W_DKV ∈ R^{d_model × d_c})

  DeepSeek-V3: d_c = 512   (vs 2×H×d_head = 32,768 in full MHA)
  Cache ONLY c_KV per token.
  Bytes per token per layer = d_c × P = 512 × 2 = 1,024 bytes = 1 KB

MLA STEP 2 — Reconstruct K,V at attention time (from cached c_KV):
  [K_1,...,K_H; V_1,...,V_H] = c_KV × W_UKV
  (W_UKV ∈ R^{d_c × (2×H×d_head)} — fixed weight, stays in SMEM/L2)

  This reconstruction is a GEMM at each attention step:
    Input:  c_KV [S_ctx × d_c]   (loaded from KV cache)
    Weight: W_UKV [d_c × 2Hd]   (fixed model weight, fits in L2)
    Output: K,V  [S_ctx × H × d_head]  (reconstructed on-the-fly in registers)

  Cost: O(S_ctx × d_c × 2Hd) extra FLOPs per attention layer.
  At S_ctx=4096, d_c=512, H=128, d_head=128:
    Extra FLOPs = 4096 × 512 × 32768 = 68.7B FLOPs
    vs attention FLOPs at S=4096: ~4×4096²×128 = 8.6B FLOPs
  Wait — this seems large. In practice: the reconstruction is amortized across all
```

```
heads.
  The actual compute vs bandwidth trade is favorable: reconstruction FLOPs are
  compute-bound; loading fewer KV bytes is bandwidth-critical for decode.

MLA STEP 3 — Decoupled RoPE for position encoding:
  Problem: if RoPE were baked into c_KV, the cached latent would be position-
specific.
  Two requests at different positions would produce different c_KV for the same
tokens.
  Prefix sharing would be impossible.

  Solution: cache a SEPARATE small k_rope component:
    k_rope = x × W_QR   (W_QR ∈ R^{d_model × d_rope}, d_rope=64)

  Full cache per token per layer:
    c_KV: 512 × 2 bytes = 1,024 bytes
    k_rope: 64 × 2 bytes = 128 bytes
    Total: 1,152 bytes = 1.125 KB per token per layer

  c_KV is position-independent → prefix-cacheable!
  k_rope carries position information needed for RoPE at query time.
```

11.7.3 MLA vs GQA vs MHA: The Complete Comparison

```
KV CACHE COMPARISON TABLE (DeepSeek-V3 config: H=128, d_head=128, BF16):

  Variant   KV bytes/tok/layer  KV at S=8K, 1 seq   Quality vs MHA
  ──────────────────────────────────────────────────────────────────
  MHA       65,536 (64 KB)       32.8 GB              Baseline
  GQA R=8    8,192 (8 KB)         4.1 GB              -0.2 to -0.5%
  GQA R=16   4,096 (4 KB)         2.1 GB              -0.3 to -0.8%
  MQA           512 (0.5 KB)       0.26 GB             -1 to -2%
  MLA         1,152 (1.125 KB)     0.58 GB             ~0% (MHA-level)
  ──────────────────────────────────────────────────────────────────

  MLA achieves GQA-level KV memory while retaining MHA-level quality.
  This is the claimed "free lunch" of MLA — empirically validated in
  DeepSeek-V2/V3 production deployments.

WHY MLA WORKS — THE LOW-RANK INTUITION:
  The concatenated KV projection matrix W_KV ∈ R^{d_model × 2Hd_head}
  has effective rank << 2×H×d_head in a trained transformer.
  Most KV information lies in a low-dimensional subspace.
```

```
  W_DKV performs implicit dimensionality reduction of this subspace.
  W_UKV reconstructs per-head diversity from the compressed representation.
  The learned factorization (W_DKV × W_UKV) is the MLA mechanism.

INFRASTRUCTURE IMPACT (DeepSeek-V3, 671B MoE, 61 layers, H=128, BF16):
  MHA KV at S=128K: 61 × 65,536 × 131,072 = 524 GB  (larger than entire H100!)
  MLA KV at S=128K: 61 × 1,152 × 131,072  = 9.2 GB  (fits on one H100)

  MLA enables 128K context serving of a 671B model on 8×H100 nodes —
  something physically impossible with standard GQA at BF16.
  This is the hardware infrastructure argument for MLA adoption.

ADOPTION TRAJECTORY:
  Currently: DeepSeek-V2/V3/R1 family.
  Likely next adopters: models targeting extreme context or low-cost serving.
  Barrier: requires architectural change at training time; no retrofit to
  existing MHA/GQA models.
```

11.8 KV Cache Decision Framework

11.8.1 ROI Priority Order

```
KV CACHE OPTIMIZATION — ROI RANKING:

  #1 Enable prefix caching: reduces prefill by 60–80% for chatbot workloads
     Free GPU time available for more decode → throughput multiplier.
     Cost: zero extra hardware; one config flag.
     When to skip: single-turn workloads with no shared prefix.

  #2 Enable FP8 KV: 2× more users per GPU = 2× throughput per dollar
     -0.2% quality (within measurement noise for most applications).
     Cost: one config flag; evaluate on production task first.
     When to skip: hardware doesn't support FP8 (A100 → use INT8).

  #3 Enable PagedAttention: 3–5× more effective KV capacity vs contiguous
     Eliminates fragmentation. Required baseline for all production systems.
     Already default in vLLM, TRT-LLM, SGLang.
     Cost: 3% attention kernel overhead.

  #4 Larger batches (more aggressive continuous batching):
     Higher GPU utilization → better throughput per dollar.
```

```
     Requires sufficient KV headroom (enabled by #1–#3 above).
     Cost: higher TPOT variance at large batch sizes.

  #5 Disaggregated P/D at fleet scale: 20–35% cost reduction
     Worthwhile at > 100 GPUs with mixed prompt/decode workloads.
     Cost: high operational complexity; KV transfer infrastructure.

  #6 INT4 KV (if quality validated): 4× KV capacity
     Only if MMLU -1.5% delta is acceptable for your specific use case.
     Evaluate on your production task before deploying.

  #7 MLA (model architecture): 57× vs BF16 MHA
     Requires training an MLA model from scratch.
     Not a retrofit option for existing deployments.
     Factor into new model selection decisions.
```

11.8.2 Technique Comparison Table

```
KV CACHE TECHNIQUE SUMMARY:

  Technique              Mem Impact   Quality Impact  TTFT Impact  Complexity
  ──────────────────────────────────────────────────────────────────────────────
  PagedAttention         3–5× eff     None            None         Required
baseline
  GQA (R=8)              8× KV        <0.5% MMLU      None         Model selection
  FP8 KV quant           2× reduction -0.2% MMLU      None         One config flag
  INT8 KV quant          2× reduction -0.5% MMLU      None         One config flag
  INT4 KV quant          4× reduction -1.5% MMLU      None         Eval required
  Prefix caching         Frees cache  None            -50–89%      One config flag
  RadixAttention         Better sharing None           -60–89%     SGLang
deployment
  CPU DRAM offloading    ×10 capacity None            +10–40ms     OS/runtime
  NVMe offloading        ×100 capacity None           +50–500ms    Complex
  P/D disaggregation     Independent  None            -30% fleet   High complexity
  MLA architecture       57× vs MHA   ~0%             Prefix-ready Model redesign
  ──────────────────────────────────────────────────────────────────────────────
```

11.8.3 Anti-Patterns

```
KV CACHE ANTI-PATTERNS — KNOW THESE FOR PRODUCTION REVIEWS:

ANTI-PATTERN 1: Over-Allocating Pages
  Symptom: Set gpu_memory_utilization=0.99 to maximize KV space.
```

```
  Problem: At >90% KV utilization, cascading evictions cause P99 TTFT spikes
           of 10-50×. The system degrades catastrophically, not gracefully.
  Correct: Set gpu_memory_utilization=0.85-0.90. Leave 10-15% headroom.
           Monitor: alert when KV utilization > 80% for > 5 minutes.

ANTI-PATTERN 2: Deploying BF16 KV in Production
  Symptom: Maximum concurrency capped at 14 users on H200 at 4K context.
  Problem: 2× memory waste with zero quality benefit vs FP8 KV.
  Correct: Enable FP8 KV by default. Re-evaluate if quality regression observed.
           For Ampere (A100): INT8 KV is the equivalent safe default.

ANTI-PATTERN 3: Ignoring Prefix Caching for Chatbot Workloads
  Symptom: TTFT = 380ms for a chatbot with 2,048-token system prompt.
  Problem: Recomputing 2,048 tokens for every request = 89% avoidable compute.
  Correct: Enable prefix caching. TTFT drops to ~34ms for cache hits.
           Measure: monitor prefix_cache_hit_rate metric; if < 50%, investigate
           whether session routing is directing turns to different servers.

ANTI-PATTERN 4: Uniform Context Length Allocation
  Symptom: Cluster sized for max context × all users (extremely over-provisioned).
  Problem: Real traffic has power-law context distribution.
           50% of requests < 1K tokens; paying for 32K capacity for all.
  Correct: Hybrid tiers. Route by expected context length.
           Short context (< 8K): high-density nodes; long context: high-memory
nodes.

ANTI-PATTERN 5: Overlooking Multi-Turn Routing
  Symptom: Prefix caching enabled but hit rate is < 20% on a chatbot.
  Problem: Load balancer routing each conversation turn to different servers.
           Each server sees the conversation for the first time → always a miss.
  Correct: Implement session-sticky routing. Route all turns in a conversation
           to the same server instance. Use conversation_id as routing key.
```

11.9 Key Takeaways and Review Questions

```
SIX KV CACHE NUMBERS TO MEMORIZE (LLaMA-3 70B GQA-8, FP8):

  #1  Per token per layer:      2 KB  (2 × 8 × 128 × 1B)
  #2  Per token all layers:     160 KB (× 80 layers)
  #3  Per sequence at 4K ctx:   0.67 GB (× 4,096 tokens)
  #4  Fragmentation (pre-paged): 80% internal waste (avg 800/4096 token use)
```

```
  #5  Prefix caching gain:       Up to 91% KV memory reduction + 89% TTFT
  #6  MLA vs BF16 MHA:           57× smaller KV (DeepSeek-V3 config)

KEY PRINCIPLES:

  1. PAGEATTENTION IS THE MANDATORY BASELINE:
     Flat contiguous KV allocation wastes 60–80% of HBM on fragmentation.
     Always use paged allocation. It is already default in vLLM, TRT-LLM, SGLang.
     The 3% attention overhead is the cheapest trade in serving engineering.

  2. FP8 KV IS ALWAYS THE CORRECT DEFAULT:
     -0.2% quality loss is within measurement noise for production applications.
     2× more concurrent users at zero hardware cost.
     One config flag. No reason not to enable it.

  3. CONCURRENCY SCALES LINEARLY WITH 1/CONTEXT_LENGTH:
     Each doubling of context halves concurrent capacity at fixed HBM.
     This is the most important equation for capacity planning.
     concurrency ∝ KV_budget_GB / (context_tokens × 160 KB)

  4. PREFIX CACHING CAN MULTIPLY CAPACITY BY 5–11×:
     For chatbot workloads: the system prompt is read-only after first request.
     With 80% hit rate on 2,048-token system prompt:
       72% fewer prefill tokens, 91% less KV memory per user.
     Requires: session-sticky routing AND prefix caching enabled.

  5. MLA IS THE ARCHITECTURAL FUTURE OF KV COMPRESSION:
     57× vs MHA BF16, ~0% quality loss.
     Enables 128K context on hardware that cannot handle standard GQA.
     Not retrofittable — requires training MLA models from scratch.
     Factor into model selection, not serving optimization.

  6. NEVER RUN KV UTILIZATION ABOVE 85%:
     Above 85%, cascading evictions create P99 TTFT cliffs.
     Set admission control at 82% KV utilization (return 429 beyond this).
     Leave headroom — KV is not like disk space where 90% full is routine.
```

Review Questions

Difficulty: Foundational

Question 1 (KV Sizing from Scratch): A new 30B parameter model has 48 transformer layers, GQA with H_q=32 and H_kv=4, d_head=128. You will serve it with FP8 KV cache on H100 SXM5 (80 GB). FP8 weights require 30 GB total. Context length is fixed at 8,192 tokens. (a) Calculate KV

bytes per token per layer. (b) Calculate KV GB per sequence at 8,192 context. (c) With 80 × 0.85 - 30 = 38 GB KV budget, how many concurrent sequences can you serve? (d) Compare to MHA (H_kv = H_q = 32) at the same context. How many concurrent seqs?

Expected answer: (a) 2 × H_kv × d_head × bytes = 2 × 4 × 128 × 1B = 1,024 bytes = 1 KB per token per layer. (b) Per sequence: 1 KB × 8,192 tokens × 48 layers = 393,216 KB = 384 MB = 0.375 GB. (c) 38 GB / 0.375 GB = 101 concurrent sequences. (d) MHA (H_kv=32): 2 × 32 × 128 × 1B = 8,192 bytes/tok/layer. Per seq: 8 KB × 8,192 × 48 = 3,145,728 KB = 3 GB. Concurrent: 38/3 = 12 sequences. GQA-4 vs MHA: 101 vs 12 = 8.4× more concurrent users from GQA alone.

Difficulty: Intermediate

Question 2 (Prefix Cache ROI Calculation): Your production RAG chatbot serves 500,000 requests/day. Each request consists of: a 4,096-token RAG document (same document for all requests in a session), a 512-token conversation history, and a 128-token user query. The RAG document changes only when the user uploads a new document (every 20 requests on average). (a) What is the effective prefix cache hit rate for the RAG document? (b) How many prefill tokens are required per day without and with prefix caching? (c) If prefill throughput is 100,000 tokens/sec on your H100, how many GPU-hours per day are saved by prefix caching?

Expected answer: (a) The RAG document (4,096 tokens) is shared for ~20 consecutive requests. Hit rate = 19/20 = 95% for the document portion. (b) Total tokens per request: 4,096 + 512 + 128 = 4,736 tokens. Without cache: 500,000 × 4,736 = 2.368B tokens/day. With 95% cache hit on 4,096-token RAG document: Cache hits (475K requests): prefill = 512 + 128 = 640 tokens. Cache misses (25K requests): prefill = 4,736 tokens. Total: 475,000 × 640 + 25,000 × 4,736 = 304M + 118.4M = 422.4M tokens. Reduction: (2,368 - 422) / 2,368 = 82.2% fewer prefill tokens. (c) Without cache: 2.368B / 100,000 = 6.58 GPU-hours/day for prefill. With cache: 422M / 100,000 = 1.17 GPU-hours/day. Saved: 5.41 GPU-hours/day. At $1.66/GPU-hr owned: $8.98/day × 365 = $3,278/year saved in compute, from one config flag.

Difficulty: Principal Level

Question 3 (System Design — High-Concurrency Chatbot): Design the KV cache architecture for a production chatbot serving 10,000 concurrent users with the following characteristics: fixed 2,048-token system prompt for all users, avg conversation history 1,024 tokens, avg user query 200 tokens, avg session length 15 turns. Hardware: 32× H100 SXM5 with FP8 weights for LLaMA-3 70B (TP=1 per instance, 70 GB weights, 10 GB KV headroom per GPU). (a) Without prefix caching, can 32 GPUs support 10,000 concurrent users? Show the math. (b) With prefix caching and 90% hit rate on the system prompt, recalculate. (c) Design the routing and memory management strategy.

Expected answer: (a) Without prefix caching: active KV per user at mid-session: 2,048 (system) + 1,024 (history) + 200 (current) = 3,272 tokens. FP8 KV: 3,272 × 160 KB = 511 MB = 0.499 GB per user. Available KV: 32 GPUs × 10 GB = 320 GB. Max concurrent users: 320 / 0.499 = 641 users. Answer: 32 H100s support only ~641 users, not 10,000. Would need ~160 H100s without caching. (b) With prefix caching (90% hit rate on 2,048-token system prompt): For cache-hit users: active KV = only conversation history + current query = 1,224 tokens. KV: 1,224 × 160 KB = 191 MB = 0.187 GB per user (system prompt is shared read-only pages). For cache-miss users (10%): full 3,272 tokens = 0.499 GB. Blended average: 0.9 × 0.187 + 0.1 × 0.499 = 0.168 + 0.050 = 0.218 GB per user. Users on 320 GB: 320 / 0.218 = 1,468 concurrent users. Better, but still short of 10,000. Need 10,000 × 0.218 / 10 = 218 H100s for 10K with caching. Additional techniques: INT8 KV (0.5×) → 0.109 GB/user → 2,936 users on same HW. MQA (64× reduction vs MHA) → extreme but not typical. Tiering: push idle conversation KV to CPU DRAM (512 GB per node → 10× overflow capacity). (c) Architecture: (1) Session-sticky routing at load balancer: route by session_id hash → same GPU group for each conversation. (2) Two-tier KV: hot KV (active decode) in GPU HBM; warm KV (between turns, < 5 min idle) in CPU DRAM. (3) Shared prefix pages for system prompt: one physical copy via PagedAttention CoW, reference-counted across all sessions. (4) Admission control at 80% KV utilization: queue new sessions rather than over-commit. (5) Monitoring: track prefix_cache_hit_rate, kv_utilization, P99 TTFT — alert on any anomaly in hit rate (often caused by routing failures that direct multi-turn sessions to different servers).

Difficulty: Principal Level

Question 4 (MLA Architecture Trade-off): Your team is selecting the base model architecture for a new product that will serve 128K context to enterprise users on H100 hardware. Candidates are: LLaMA-3-style GQA (H_q=64, H_kv=8) and a DeepSeek-MLA-style model (d_c=512, d_rope=64, H=64, d_head=128). Both are trained at equal quality (same MMLU). For LLaMA-3 style use L=80 layers; for MLA use L=61 layers (DeepSeek-V3 architecture). BF16 weights throughout. (a) Compare KV cache GB per sequence at S=128K for both architectures. (b) What are the concurrent user implications on one H200 (141 GB)? (c) What is the key infrastructure advantage and the key limitation of choosing MLA?

Expected answer: (a) GQA (H_kv=8, d_head=128, L=80, BF16): 2 × 8 × 128 × 2B × 80 layers = 327,680 bytes/tok. At S=128K: 131,072 × 327,680 = 42.9 GB per sequence. MLA (d_c=512, d_rope=64, L=61, BF16): (512 + 64) × 2B × 61 layers = 70,272 bytes/tok = 68.6 KB/tok. At S=128K: 131,072 × 70,272 = 9.2 GB per sequence. Ratio: 42.9 / 9.2 = 4.7× lower KV with MLA. (b) H200 (141 GB), BF16 weights: GQA weights ≈ 2× 70B × 2B = 140 GB → barely any KV headroom! Would need FP8 weights (70 GB) to leave 71 GB for KV. At FP8 weights: 71 / 42.9 = 1.65 → can barely fit 1 sequence. With MLA (140 GB BF16 weights): same problem. With FP8 MLA weights: 71 / 9.2 = 7.7 → can serve 7 concurrent 128K sessions. MLA with FP8 weights gives 7 concurrent users; GQA with FP8 weights gives 1. (c) Key infrastructure advantage: MLA enables 128K+ context serving on hardware that cannot physically support standard GQA at the same context. At S=128K, GQA

requires dedicating 42.9 GB of KV per user — making multi-user serving impractical on any single GPU. MLA's 9.2 GB enables real concurrent serving. For an enterprise product where users routinely need 100K token context, MLA is the enabling technology, not a marginal improvement. Key limitation: MLA requires training a new model from scratch — no existing LLaMA, Mistral, or Gemma checkpoints can be converted to MLA post-hoc. The reconstruction weight matrices W_UKV are learned jointly with the model. This means: cannot leverage pre-trained open weights; requires full pretraining compute; the ecosystem (LoRA adapters, quantized weights, fine-tuned variants) must be rebuilt from scratch on the MLA base model.

Further Reading

1. *Efficient Memory Management for Large Language Model Serving with PagedAttention* — Kwon et al. (2023). `arxiv.org/abs/2309.06180` The foundational vLLM paper. Read Section 3 (memory management) and Section 4 (system design).
2. *SGLang: Efficient Execution of Structured Language Model Programs* — Zheng et al. (2024). `arxiv.org/abs/2312.07104` RadixAttention algorithm and the radix tree eviction policy. Section 3.2.
3. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model* — DeepSeek AI (2024). `arxiv.org/abs/2405.04434` The original MLA paper. Read Section 2 for the full formulation. The low-rank KV compression derivation is in Section 2.1.2.
4. *DeepSeek-V3 Technical Report* — DeepSeek AI (2024). `arxiv.org/abs/2412.19437` MLA at 671B scale, FP8 training, inference infrastructure. Section 3 covers serving.
5. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting* — Patel et al. (2024). `arxiv.org/abs/2311.18677` The disaggregated P-D paper with KV transfer quantification.
6. *NVIDIA Dynamo: Inference Serving at Scale* — NVIDIA Technical Blog (2025). `developer.nvidia.com/blog/nvidia-dynamo` NIXL KV transfer protocol, layer-pipelined KV transfer mechanics.

— End of Chapter 11 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved